MBINGULA – Guia completo de instalação e arranque

Backend em Node.js (Express + Prisma) com base de dados PostgreSQL e frontend em React. Guia escrito para Windows, passo a passo, para quem nunca viu o projecto.

1. Visão geral

O sistema tem três peças que têm de estar a funcionar ao mesmo tempo:

Peça	Tecnologia	Onde corre
Base de dados	PostgreSQL	localhost, porta 5432
Backend (API)	Node.js, Express, Prisma	http://localhost:3000
Frontend	React	normalmente http://localhost:3000 ou 5173 (ver secção 12)

O backend está na pasta `MBINGULA_BACKEND_API-main`. A ordem correcta é sempre: **1) PostgreSQL ligado, 2) backend a correr, 3) frontend a correr**.

2. Programas necessários

Instala estes programas antes de começar:

1. **Node.js (versão LTS)** – https://nodejs.org . Instala também o `npm`. Aceita as opções por defeito.
2. **PostgreSQL** – https://www.postgresql.org/download/windows . Durante a instalação:
   - guarda a **palavra-passe** do utilizador `postgres` (neste projecto: `celso123`);
   - deixa a porta **5432**;
   - deixa o **pgAdmin** seleccionado (interface gráfica da base de dados).
3. **Visual Studio Code** – https://code.visualstudio.com
4. **Extensão Thunder Client** (dentro do VS Code, separador Extensões, pesquisar `Thunder Client`). Serve para enviar pedidos à API sem escrever código.
5. **Git** (opcional, só se fores descarregar o projecto do GitHub) – https://git-scm.com

Para confirmar que o Node ficou instalado, abre o PowerShell e corre:

```
node -v
npm -v
```

Deves ver números de versão. Se aparecer erro, fecha e volta a abrir o PowerShell (ou reinicia o computador).

3. Abrir o projecto

1. Abre o VS Code e vai a **File > Open Folder**.
2. Escolhe a pasta do backend: `C:\Users\Abençoado\Documents\GitHub\mbingula\MBINGULA_BACKEND_API-main`
3. Abre um terminal dentro do VS Code (**Terminal > New Terminal**). Todos os comandos seguintes são corridos **nesta pasta**, ao lado do ficheiro `package.json`.

Se o terminal estiver noutra pasta, entra na certa com:

```
cd C:\Users\Abençoado\Documents\GitHub\mbingula\MBINGULA_BACKEND_API-main
```

4. Instalar as dependências do backend

```
npm install
```

Isto descarrega todas as bibliotecas (Express, Prisma, cors, multer, pino-http, etc.) para a pasta `node_modules`. Só é preciso repetir se apagares essa pasta ou se o `package.json` mudar.

Para ver quais os comandos (scripts) que o projecto tem:

```
npm run
```

Nota: o comando `npm run migrate` **não existe** neste projecto (dá o erro `Missing script: migrate`). As migrações fazem-se com o Prisma, na secção 7.

5. Garantir que o PostgreSQL está a correr

No PowerShell:

```
Get-Service *postgres*
```

- Se o estado for `Running`, está tudo bem.

- Se for `Stopped`, abre o PowerShell **como Administrador** e corre (usa o nome exacto do serviço que apareceu; o número 16 pode ser outro):

```
Start-Service postgresql-x64-16
```

Alternativa: `Win + R`, escrever `services.msc`, procurar `postgresql`, botão direito, **Iniciar**.

Se o comando não mostrar nenhum serviço, o PostgreSQL não está instalado (volta à secção 2).

6. O ficheiro `.env`

Onde fica

Na **raiz do backend**, na mesma pasta do `package.json`:

```
MBINGULA_BACKEND_API-main\
├── package.json
├── .env            <-- aqui
├── prisma\
│   └── schema.prisma
└── src\
```

O ficheiro chama-se apenas `.env` (começa com ponto, sem nome antes e sem `.txt` no fim). Normalmente **não vem do GitHub** (está no `.gitignore`), por isso tens de o criar se não existir.

Conteúdo

```
DATABASE_URL = "postgresql://postgres:celso123@localhost:5432/prisma_db?
schema=public"
MY_SECRET_KEY = "546797867treygmhngfdty4ey66utr6545y6u9o"
```

O que significa cada parte

Parte	Significado
`postgres` (antes dos dois pontos)	utilizador do PostgreSQL
`celso123`	palavra-passe do utilizador `postgres` (tem de ser a que definiste na instalação)
`localhost`	a base de dados está neste computador

Parte	Significado
`5432`	porta do PostgreSQL
`prisma_db`	nome da base de dados (é criada automaticamente pelo Prisma)
`schema=public`	esquema padrão do PostgreSQL
`MY_SECRET_KEY`	chave secreta usada pelo sistema de login para assinar os tokens

Regras importantes

- Se a palavra-passe do teu PostgreSQL for outra, troca `celso123` pela correcta.
- Evita os caracteres `@`, `#`, `/` e `:` na palavra-passe, porque quebram o URL.
- Depois de alterares o `.env`, **reinicia** a API.
- Nunca partilhes este ficheiro nem o envies para o GitHub. Em produção, usa uma palavra-passe e uma `MY_SECRET_KEY` novas.

7. Criar as tabelas da base de dados (migração)

Com o PostgreSQL ligado e o `.env` criado:

```
npx prisma migrate dev
```

O Prisma lê o ficheiro `prisma\schema.prisma`, cria a base `prisma_db` se não existir e cria todas as tabelas. Se pedir um nome para a migração, escreve qualquer um (por exemplo `nova migracao`).

Resultado esperado:

```
PostgreSQL database prisma_db created at localhost:5432
Applying migration ...
Your database is now in sync with your schema.
Generated Prisma Client
```

Se o Prisma Client não for gerado, corre à mão:

```
npx prisma generate
```

Tabelas criadas

User, UserAccess, Access, Sector, Funcionario, ItemCardapio, Image, Avaliacao, Servicorestaurante, Fornecedor, Produto, Estoque, Mesa, Atividadecasa, PedidoLocal e PedidoCasa.

Ver a base de dados num ecrã

```
npx prisma studio
```

Abre em http://localhost:5555 e mostra todas as tabelas e registos. Para fechar, carrega em `Ctrl + C` no terminal.

Erros comuns nesta fase

Erro P1001: Can't reach database server at localhost:5432 O PostgreSQL não está ligado ou não está instalado. Volta à secção 5.

Erro P1000 / credenciais inválidas A palavra-passe do `.env` não é a do PostgreSQL. Testa a ligação com:

```
psql -U postgres -h localhost
```

Se o `psql` não for reconhecido, usa o caminho completo, por exemplo `& "C:\Program Files\PostgreSQL\16\bin\psql.exe" -U postgres -h localhost`.

Redefinir a palavra-passe do `postgres` (se a esqueceste)

1. Abre como Administrador o ficheiro `C:\Program Files\PostgreSQL\16\data\pg_hba.conf`.
2. Nas linhas `127.0.0.1/32` e `::1/128`, muda `scram-sha-256` (ou `md5`) para `trust`. Guarda.
3. Reinicia o serviço (PowerShell como Administrador): `Restart-Service postgresql-x64-16`
4. Entra sem palavra-passe: `psql -U postgres -h localhost`
5. Define a nova: `ALTER USER postgres WITH PASSWORD 'nova_palavra_passe';` e sai com `\q`.
6. **Volta a pôr `scram-sha-256`** no `pg_hba.conf`, guarda e reinicia o serviço outra vez.
7. Actualiza o `.env` com a nova palavra-passe.

8. Arrancar a API

```
npm run dev
```

Se der `Missing script`, vê com `npm run` quais os scripts existentes (pode ser `npm start`). Deve aparecer a mensagem **`Servidor iniciou`**. A API fica em:

```
http://localhost:3000
```

O terminal fica ocupado enquanto a API corre; **não o fechar**. Para parar, `Ctrl + C`. Cada pedido recebido aparece no terminal em formato JSON (biblioteca `pino-http`), útil para ver erros.

As imagens dos itens do cardápio ficam na pasta `uploads` e são servidas em `http://localhost:3000/images/nome-do-ficheiro`.

9. Usar o Thunder Client

1. Clica no ícone do Thunder Client na barra lateral do VS Code.
2. **New Request**.
3. Escolhe o método (GET, POST, PUT, PATCH, DELETE) e escreve o URL.
4. Para pedidos com dados (POST/PUT/PATCH): separador **Body**, opção **JSON**, e escreve o JSON.
5. Clica em **Send** e lê o resultado (código de estado e resposta).

Significado dos códigos: **200/201** = sucesso; **400** = dados inválidos ou em falta (a mensagem explica); **401/403** = sem permissão ou sem login; **404** = rota errada; **500** = erro no servidor (ver o terminal da API).

10. Criar os acessos (papéis) e o primeiro utilizador

A ordem é obrigatória: **primeiro os acessos, depois o utilizador**, porque cada utilizador tem de ter um acesso que já exista.

Passo 1 – Criar os acessos

POST para `http://localhost:3000/createAccess` com o corpo JSON:

```
{ "name": "adm" }
```

Repete o pedido com:

```
{ "name": "normal" }
```

O sistema reconhece estes dois nomes nas rotas protegidas: **adm** (administrador) e **normal**. Para confirmar, GET `http://localhost:3000/accessess`.

Passo 2 – Criar o utilizador

POST para `http://localhost:3000/user` com o corpo JSON:

```
{
  "name": "Celso",
  "email": "celso@exemplo.com",
  "phone": "923000000",
  "password": "123456",
  "accessName": "adm"
}
```

Regras:

- **Todos os cinco campos são obrigatórios**, incluindo `email` e `phone` (mesmo que o schema os marque como opcionais, o código usa-os numa pesquisa e sem eles dá `PrismaClientValidationError`).
- O campo do papel chama-se **`accessName`** (não `access`).
- `accessName` tem de ser um papel já criado no passo 1; senão a resposta é `Este nivel de acesso não existe`.
- `email` e `phone` não podem repetir-se. Mensagens possíveis: `Já existe um usuário com este email` e `Este número de telefone já existe`.
- A palavra-passe é guardada **tal como a escreves** (sem encriptação).

Sucesso: resposta 200 com o utilizador e o seu papel dentro de `userAccess`.

Passo 3 – Verificar

GET `http://localhost:3000/getAllUsers` mostra os utilizadores. Também se pode ver no `npx prisma studio`.

Mudar o acesso de um utilizador

PATCH/PUT conforme o router: `PUT http://localhost:3000/update-userAccess/ID_DO_UTILIZADOR` com `{ "accessName": "normal" }`.

11. Login

POST `http://localhost:3000/sign-in`. Os campos prováveis são `email` e `password`, mas confirma no ficheiro `src/controller/SessionController.ts`. Se o login funcionar, devolve um **token**. Para usar rotas protegidas (por exemplo `/getLoginUser`), no Thunder Client vai a **Auth > Bearer** e cola o token (ou cria o header `Authorization: Bearer TOKEN`).

Aviso: como a palavra-passe é gravada sem encriptação, se o `SessionController` usar `bcrypt.compare` o login falhará para utilizadores criados assim. Nesse caso é preciso ajustar o código (guardar com `bcrypt.hash` no `createUser`).

12. Frontend em React

O frontend está noutra pasta/repositório (não faz parte da pasta do backend). Passos gerais:

```
cd caminho\da\pasta\do\frontend
npm install
npm start
```

(ou `npm run dev` se usar Vite; vê com `npm run`).

- O frontend deve apontar para a API em `http://localhost:3000`. Procura no código do frontend onde está o URL da API (ficheiro `api.js` / `api.ts`, `services` ou um `.env` do frontend, por exemplo `REACT_APP_API_URL` ou `VITE_API_URL`) e confirma que é este.
- Como o backend já usa `cors()`, aceita pedidos vindos de outra porta.
- **Conflito de porta:** a API usa a 3000, que é também a porta por defeito do React (`npm start`). Se o React perguntar se queres usar outra porta, responde **Y**.
- O frontend precisa da API ligada e de pelo menos um utilizador criado para fazer login.

13. Rotas principais da API

Todas começam em `http://localhost:3000`, sem prefixo.

Área	Rotas
Utilizador	POST /user, GET /getAllUsers, GET /get-user/:id, GET /getUserByName/:name, PUT /update-user/:id, PUT /update-userAccess/:id, DELETE /delete-user/:id, DELETE /delete-users

Área	Rotas
Acesso	GET /accessess, GET /accessById/:id, POST /createAccess
Login	POST /sign-in, GET /getLoginUser (com token)
Sector	POST /create-sector, GET /sectores, GET /sectorById/:id, PUT /update-sector/:id, DELETE /delete-sector/:id
Funcionário	POST /create-funcionario, GET /getAllFuncionarios, GET /get-funcionario/:id, PUT /update-funcionario/:id, DELETE /delete-funcionario/:id
Item do cardápio	POST /create-ItemCardapio (com ficheiro), GET /getItemsCardapio, GET /getItemById/:id, PUT /update-itemCardapio/:id, DELETE /delete-itemCardapio/:id
Avaliação	POST /create-avaliacao, GET /getAvaliacoes, PUT /update-avaliacao/:id, DELETE /delete-avaliacao/:id
Serviço	POST /create-servico, GET /getServico, PUT /update-servico/:id, DELETE /delete-servico/:id
Fornecedor	POST /create-fornecedor, GET /getAllFornecedor, PUT /update-fornecedor/:id, DELETE /delete-fornecedor/:id
Produto	POST /create-produto, GET /getAllProduto, PUT /update-produto/:id, DELETE /delete-produto/:id
Estoque	POST /create-estoque, GET /getAllEstoques, PUT /update-estoque/:id, DELETE /delete-estoque/:id
Mesa	POST /create-mesa, GET /getAllMesas, PUT /update-mesa/:id, DELETE /delete-mesa/:id
Reserva de mesa	PATCH /create-reserva/:numero, GET /getAllAvailableTable, GET /getAllNotAvailableTable, PATCH /makeMesaAvailable-mesa/:id
Atividades	POST /create-atividade, GET /getAllAtividades, PUT /update-atividade/:id, DELETE /delete-atividade/:id

Área	Rotas
Pedido local	POST /create-pedidoLocal, GET /getAllPedidoLocal, PUT /updatePedidoLocal/:id, PATCH /updateStatusPedidoLocal/:id, DELETE /deletePedidoLocal/:id

14. Campos de cada tabela (para criar registos)

Os nomes abaixo vêm do `schema.prisma`. Os campos exactos que cada controller aceita devem ser confirmados no ficheiro correspondente em `src/controller`.

Tabela	Campos
Sector	name (obrigatório), descricao
Funcionario	name, morada, bilheteidentidade, telefone, cargo, salario (todos obrigatórios), sectorId (ligação a um Sector)
ItemCardapio	name, price, categoria, disponivel (por defeito `Sim`), imagens no campo de ficheiro `Image`
Avaliacao	assunto, descricao
Servicorestaurante	tipo, descricao
Fornecedor	nome, telefone, email, descricao
Produto	nome, descricao, fornecedorId
Estoque	quantidade, unidade, produtoId
Mesa	lugares, posicao (o número é automático; estado por defeito `Disponível`)
Atividadecasa	tema, data, hora, descricao
PedidoLocal	tipoConsumo, valorTotal, itemsPedido, metodoPagamento, status, numeroMesa, userName, userPhone

Exemplos:

```
{ "name": "Cozinha", "descricao": "Equipa da cozinha" }
```

```
{ "lugares": 4, "posicao": "Varanda" }
```

Para o item do cardápio, no Thunder Client usa **Body > Form** (multipart), com os campos de texto e um campo do tipo ficheiro chamado `Image`.

15. Resumo rápido (ordem de comandos)

```
# 1. Instalar Node.js e PostgreSQL (guardar a palavra-passe do postgres)
# 2. Entrar na pasta do backend
cd C:\Users\Abençoado\Documents\GitHub\mbingula\MBINGULA_BACKEND_API-main

# 3. Criar o ficheiro .env (conteúdo na secção 6)

# 4. Instalar dependências
npm install

# 5. Criar a base de dados e as tabelas
npx prisma migrate dev

# 6. Arrancar a API
npm run dev

# 7. (noutro terminal) Ver a base de dados
npx prisma studio
```

Depois, no Thunder Client: criar os acessos `adm` e `normal` (secção 10), criar o utilizador e fazer login.

16. Resolução de problemas

Problema	Causa	Solução
`Missing script: "migrate"`	o script não existe	usar `npx prisma migrate dev`
`P1001 Can't reach database server`	PostgreSQL desligado	`Start-Service postgresql-x64-16` (Administrador)
`P1000` credenciais inválidas	palavra-passe errada no `.env`	corrigir o `.env` ou redefinir a palavra-passe (secção 7)

Problema	Causa	Solução
`Environment variable not found: DATABASE_URL`	`.env` não existe ou está na pasta errada	colocar o `.env` ao lado do `package.json`
`PrismaClientValidationError` ao criar utilizador	faltam campos (`email`, `phone`, `accessName`)	enviar o JSON completo da secção 10
`Este nivel de acesso não existe`	papel não criado	criar `adm` / `normal` com POST /createAccess
`EADDRINUSE: port 3000`	já há algo na porta 3000	fechar a outra instância (`Ctrl + C`) ou mudar a porta em `listen`
`Cannot find module`	dependências não instaladas	`npm install`
404 no Thunder Client	rota ou porta errada	confirmar o URL na secção 13
Frontend não consegue falar com a API	API desligada ou URL errado	arrancar a API e verificar o URL no frontend

17. Segurança (antes de pôr online)

- Garante que o `.gitignore` tem a linha `.env`.
- Troca a palavra-passe da base de dados e a `MY_SECRET_KEY` por valores novos e longos.
- Guarda as palavras-passe dos utilizadores com `bcrypt` em vez de texto simples.
- Não deixes o `pg_hba.conf` em modo `trust`.
- Faz cópias de segurança regulares da base de dados (pgAdmin, botão direito na base, **Backup**).